Dictionaries / Hashing

Dictionary ADT

- The dictionary ADT models a searchable collection of key-element items
- The main operations of a dictionary are searching, inserting, and deleting items
- Multiple items with the same key are allowed

- Applications:
  - Log File
  - address book

- Dictionary ADT methods:
  - find(k): if the dictionary has an item with key k, returns the position of this item, else, returns a null position.
  - insertItem(k, o): inserts item (k, o) into the dictionary
  - removeElement(k): removes the item with key k from the dictionary. If no such element exists, an error occurs.
  - size(), isEmpty()
  - keys(), Elements()

Log File

- A log file is a dictionary implemented by means of an unsorted sequence

- Performance:
  - insertItem takes $O(1)$ time since we can insert the new item at the beginning or at the end of the sequence
  - find and removeElement take $O(n)$ time since in the worst case (the item is not found) we traverse the entire sequence to look for an item with the given key

- The log file is effective only for dictionaries of small size or for dictionaries on which insertions are the most common operations, while searches and removals are rarely performed.

Lookup Table

- A lookup table is a dictionary implemented by means of a sorted sequence
  - Store the items of the dictionary in an array-based sequence, sorted by key
- The lookup table is effective only for dictionaries of small size or for dictionaries on which searches are the most common operations (e.g., credit card authorizations).

Hash Tables

- Hash table:
  - Given a table *T* and a record *x*, with key (= symbol) and data, one needs to support:
    - Search(*T*, *x*)
    - Insert (*T*, *x*)
    - Delete (*T*, *x*)
  - These operations need to be fast.

Applications of hash tables

- Compilers use hash tables (symbol table) to keep track of declared variables.

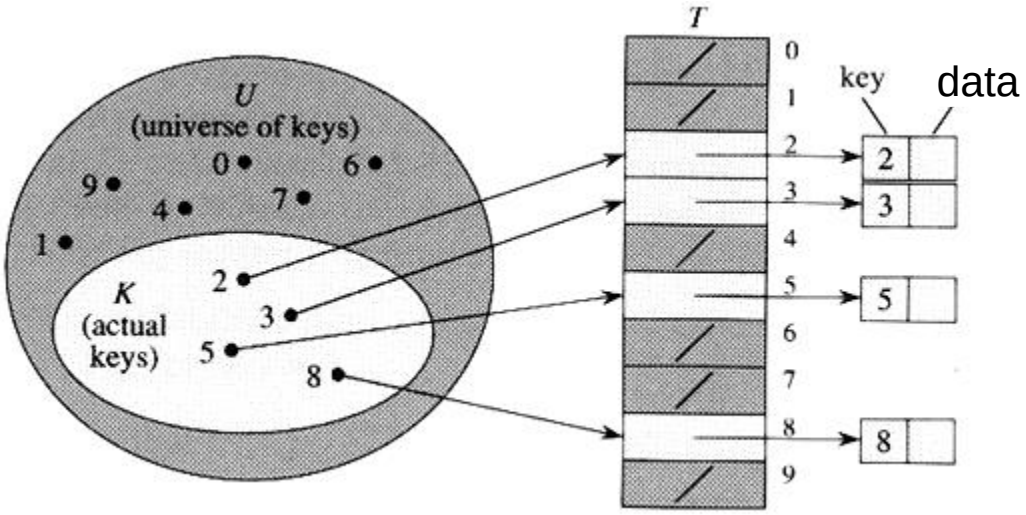
- On-line spell checkers: After pre-hashing the entire dictionary, one can check each word in constant time and print out the misspelled word in order of their appearance in the document.

- Password table, Driver's license record…

Direct Addressing

- Suppose:
  - The range of keys is 0..*m*-1
  - Keys are distinct

- The idea: Set up an array *A*[0..m-1] in which
  - *A*[i] = *x* key[*x*] = *i*
  - *A*[i] = NULL otherwise
  - Operations take *O*(1) time!



The Problem With Direct Addressing

- Direct addressing works well when the range m of keys is relatively small.
- If the keys are 32-bit integers:
  - Problem 1: direct-address table will have 2^{32} entries.

    (E.g. student IDs are 9-digit integers, so the universe size is 10^9, but we only have about 500 students)
  - Problem 2: even if memory is not an issue, the time to initialize the elements to NULL may be an issue

- Solution: map keys to smaller range 0..m-1
- This mapping is called *hash function*

Hash Functions and Hash Tables

- A hash function h maps keys of a given type to integers in a fixed interval $[0, N - 1]$
- Example:
  $h(x) = x \bmod N$
  is a hash function for integer keys
- The integer $h(x)$ is called the hash value of key x

- A hash table for a given key type consists of
  - Hash function h
  - Array (table) of size N
- When implementing a dictionary with a hash table, the goal is to store item (k, o) at index $i = h(k)$

Hash Functions

- A hash function is usually (when keys are not integers) specified as the composition of two functions :

  Hash code map: h_1: keys $\rightarrow$ integers

  Compression map: h_2: integers $\rightarrow$ [0, N - 1]

- The hash code map is applied first, and the compression map is applied next on the result,

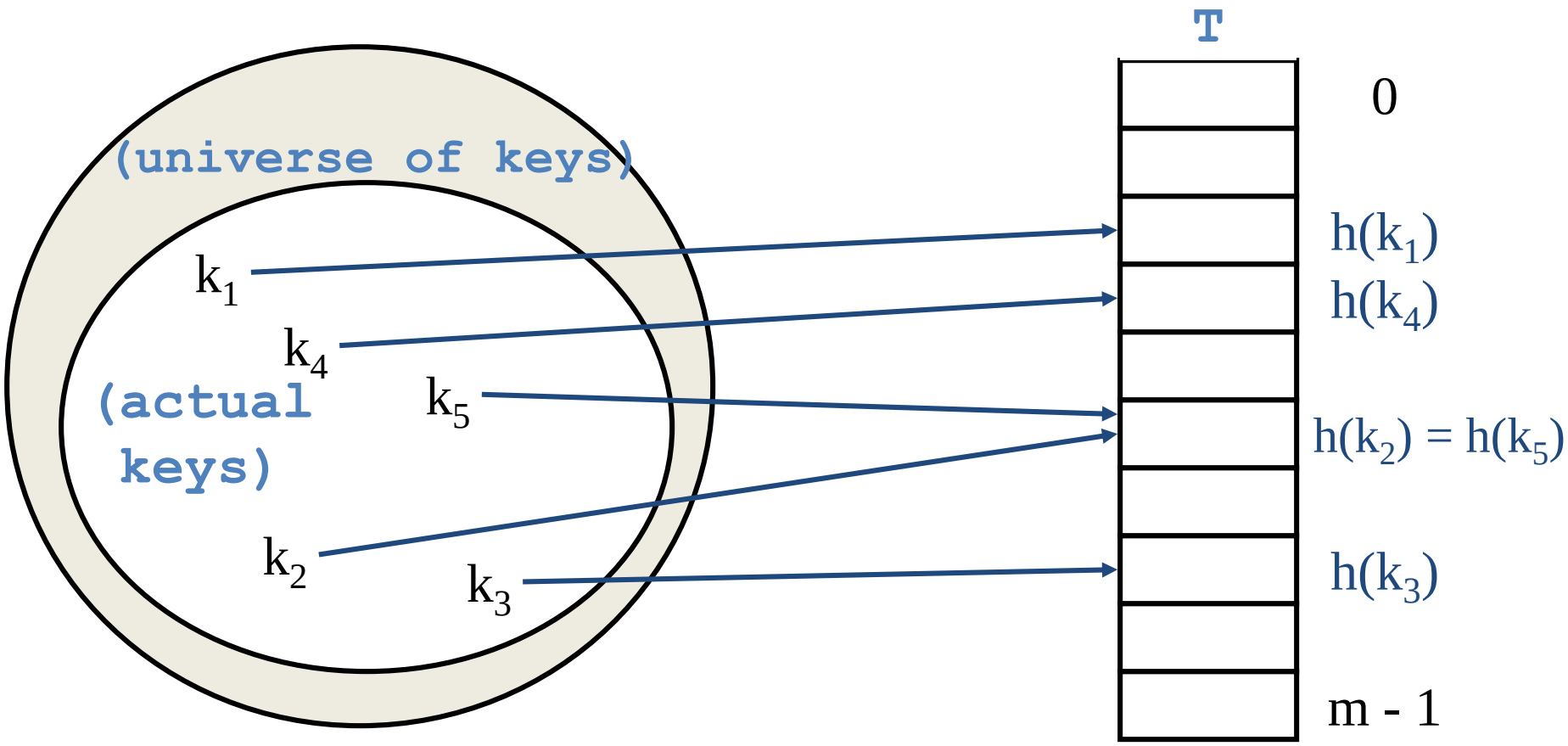
  $h(x) = h_2(h_1(x))$

Compression Maps

- Compute hash using Division method:
  - $h(y) = y \bmod N$
  - The size N of the hash table is usually chosen to be a prime

- Multiply, Add and Divide method:
  - $h_2(y) = (ay + b) \bmod N$
  - a and b are nonnegative integers such that $a \bmod N \neq 0$
  - Otherwise, every integer would map to the same value b

Hash Functions Collision



Collisions

- A collision occurs when two different keys hash to the same value.

- Example: hash table with 13 buckets
  h(k) = k % 13

  80 $\rightarrow$ 2 (80%13= 2), 40 $\rightarrow$ 1, 54 $\rightarrow$ 2 collision!

Properties of good hash functions

- Should be efficiently computable – O(1) time
- Should not waste space unnecessarily
  - For every index, there is at least one key that hashes to it
- Should minimize collisions

Collisions Handling

- Two different methods for collision resolution
  - Open hashing (Separate chaining)
  - Closed hashing (Open addressing)

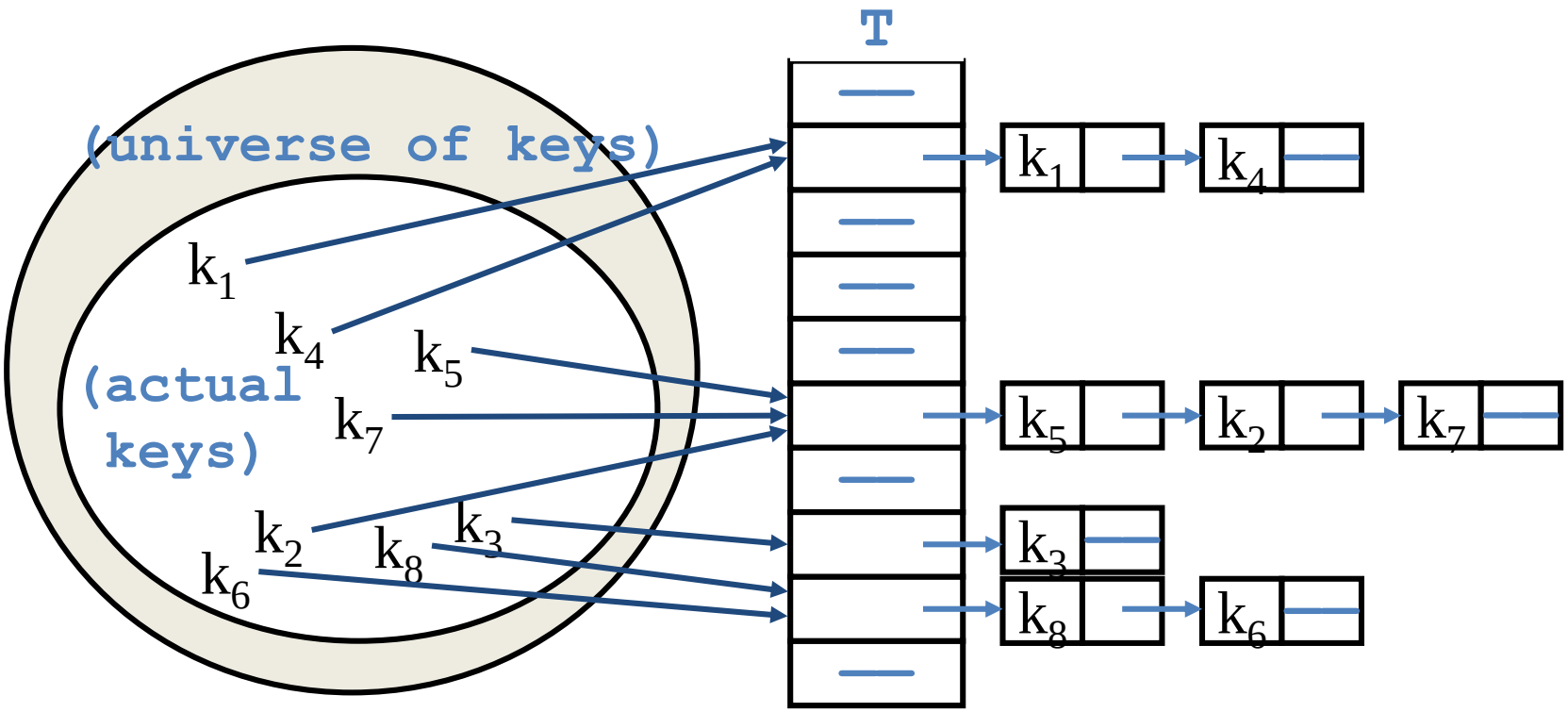
- Difference has to do with whether collisions are stored *outside the table* (open hashing) or whether collisions result in storing one of the elements at *another slot in the table* (closed hashing).

Chaining

- Each bucket in the hash table is the head of a linked list
- All elements that hash to a particular bucket are placed on that bucket's linked list
- Elements within a bucket can be ordered in several ways
  - by order of insertion, by key value order, or by frequency of access order

Chaining

- Chaining puts elements that hash to the same slot in a linked list:



Analysis of Chaining

- Assume *simple uniform hashing* : each key in table is equally likely to be hashed to any slot

- Given *n* keys and *m* slots in the table: the *load factor* α = *n*/*m* = average # keys per slot

- What will be the average cost of a successful search?
- Answer: $O(1 + \alpha)$

Open Addressing

- Basic idea:
  - To insert: if slot is full, try another slot, …, until an open slot is found (*probing*)
  - To search, follow same sequence of probes as would be used when inserting the element
    - If you reach element with correct key, return it
    - If you reach a NULL pointer, element is not in table
- Good for fixed sets (adding but no deletion)

Open Addressing

- Given an item X, try cells $h_0(X)$, $h_1(X)$, $h_2(X)$, …, $h_i(X)$
  - $h_i(X)$ = (Hash(X) + F(i)) mod *TableSize*
  - Define F(0) = 0

- F is the *collision resolution* function
  - Linear: F(i) = i
  - Quadratic: F(i) = i^2
  - Double Hashing: F(i) = i * $Hash_2(X)$

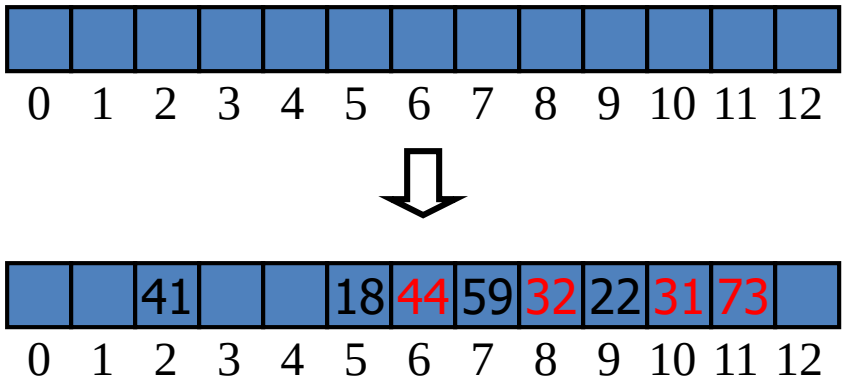
Open Addressing: Linear Probing

- Main Idea: When collision occurs, scan down the array one cell at a time looking for an empty cell
  - $h_i(X)$ = (Hash(X) + i) mod *TableSize* (i = 0, 1, 2, …)
  - Compute hash value and increment it until a free cell is found

Linear Probing

- Open addressing: the colliding item is placed in a different cell of the table
- **Linear probing** handles collisions by placing the colliding item in the next (circularly) available table cell
- Each table cell inspected is referred to as a "probe"

- Example:
  - $h(x) = x \bmod 13$
  - Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

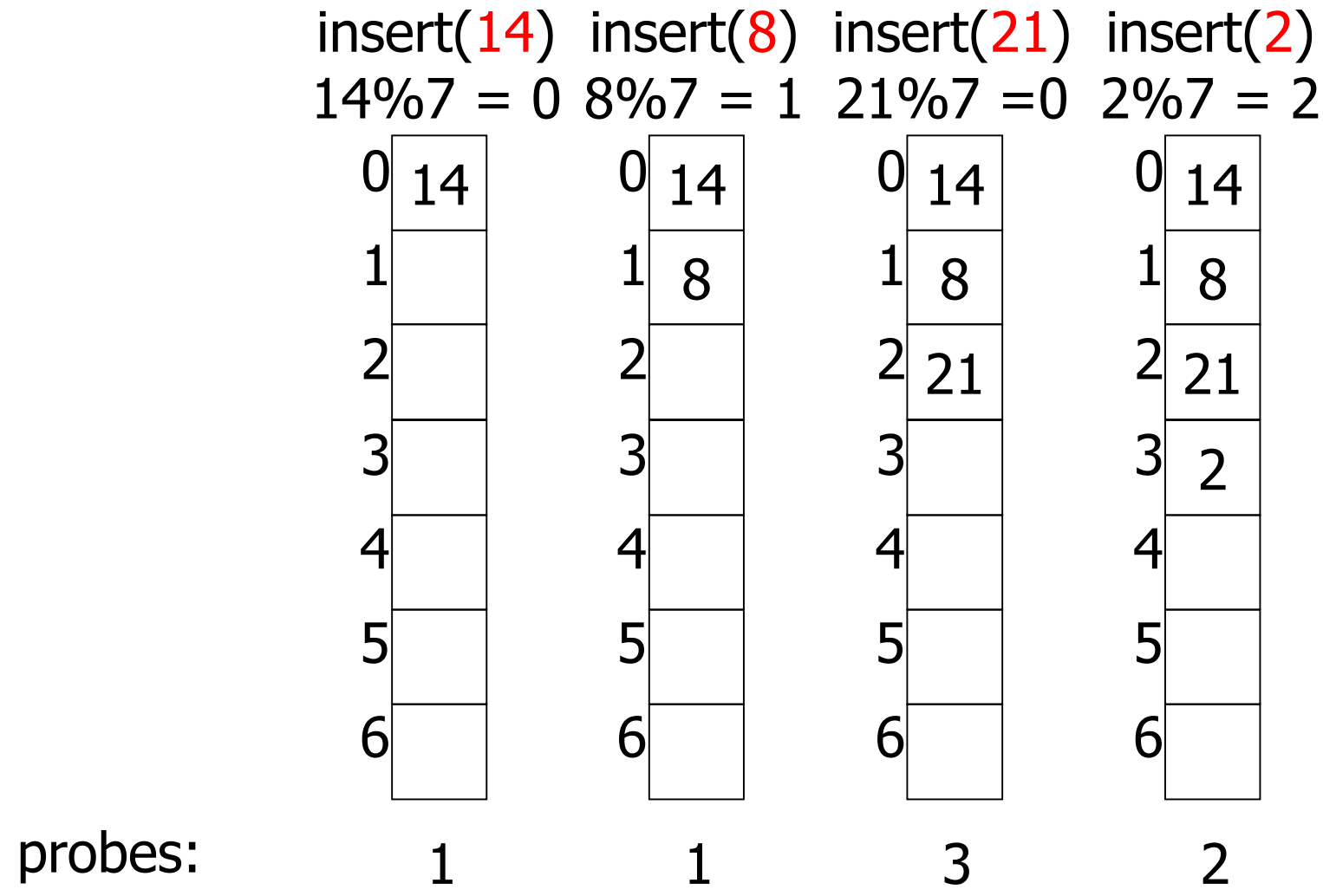


Search with Linear Probing

- Consider a hash table A that uses linear probing
- find(k)
  - We start at cell $h(k)$
  - We probe consecutive locations until one of the following occurs
    - An item with key k is found, or
    - An empty cell is found, or
    - N cells have been unsuccessfully probed

```
Algorithm find(k)
    i ← h(k)
    p ← 0
    repeat
        c ← A[i]
        if c = ∅
            return Position(null)
        else if c.key () = k
            return Position(c)
        else
            i ← (i + 1) mod N
            p ← p + 1
    until   p = N
    return Position(null)
```

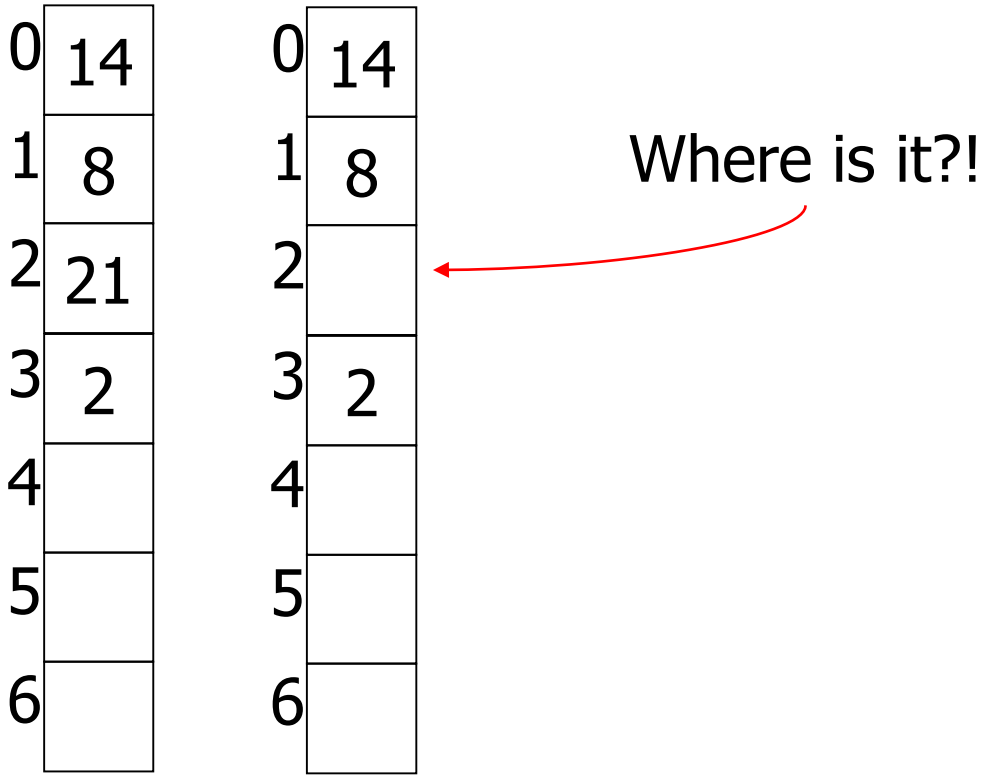
Linear Probing Example



Linear Probing Example

delete(21) find(2)

21%7 = 0 2%7 = 2



Drawbacks of Linear Probing

Works until array is full, but as number of items N approaches *TableSize*, access time approaches O(N)